HTTP vs HTTPS, TLS, HTTP/2 & CORS

Why it matters

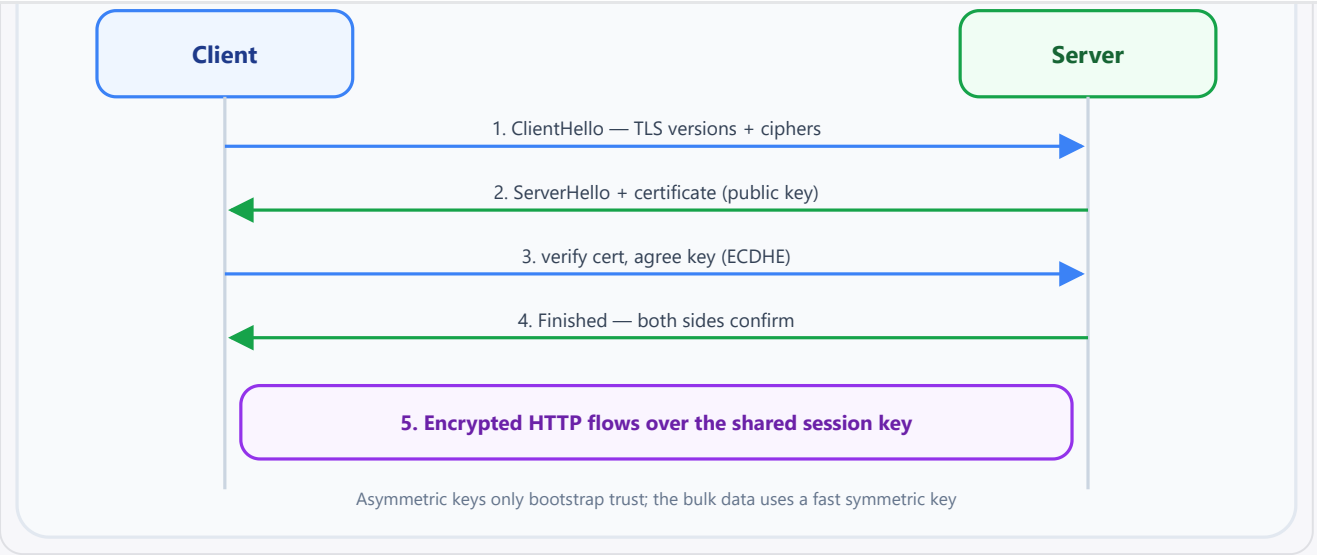
Everything so far assumed a plain, unencrypted connection. In production that is unacceptable: an unencrypted HTTP request travels in clear text, so anyone on the network path — a coffee-shop router, an ISP, a compromised switch — can read passwords and tokens or silently tamper with responses. **HTTPS** is HTTP wrapped in **TLS**, and it is the default for the entire modern web. Browsers now mark plain HTTP as "Not Secure," refuse powerful APIs over it, and increasingly auto-upgrade to HTTPS.

As a backend engineer you need to know what TLS actually guarantees (and what it does not), how Node serves HTTPS and HTTP/2, why keep-alive matters for throughput, and how CORS — a browser rule that trips up nearly every developer — interacts with your headers. These topics come up constantly in interviews as "what happens when you type a URL and press enter" and "why is my fetch blocked by CORS."

The concept, simply

HTTP is the request/response protocol. **HTTPS** is the exact same protocol running *inside* an encrypted TLS tunnel. TLS gives you three things: **confidentiality** (eavesdroppers see only ciphertext), **integrity** (tampering is detected), and **authentication** (a certificate proves you are really talking to `example.com`, not an impostor). The application code barely changes — you still write `req` / `res` — but the bytes on the wire are now encrypted.

The one-time cost is a **handshake** at the start of the connection, where client and server agree on encryption keys. After that, data flows over a fast symmetric cipher. Reusing the connection for many requests (**keep-alive**) amortises that handshake, which is why it matters for performance.



The handshake bootstraps trust with asymmetric keys, then bulk data uses a fast shared symmetric key.

How it works

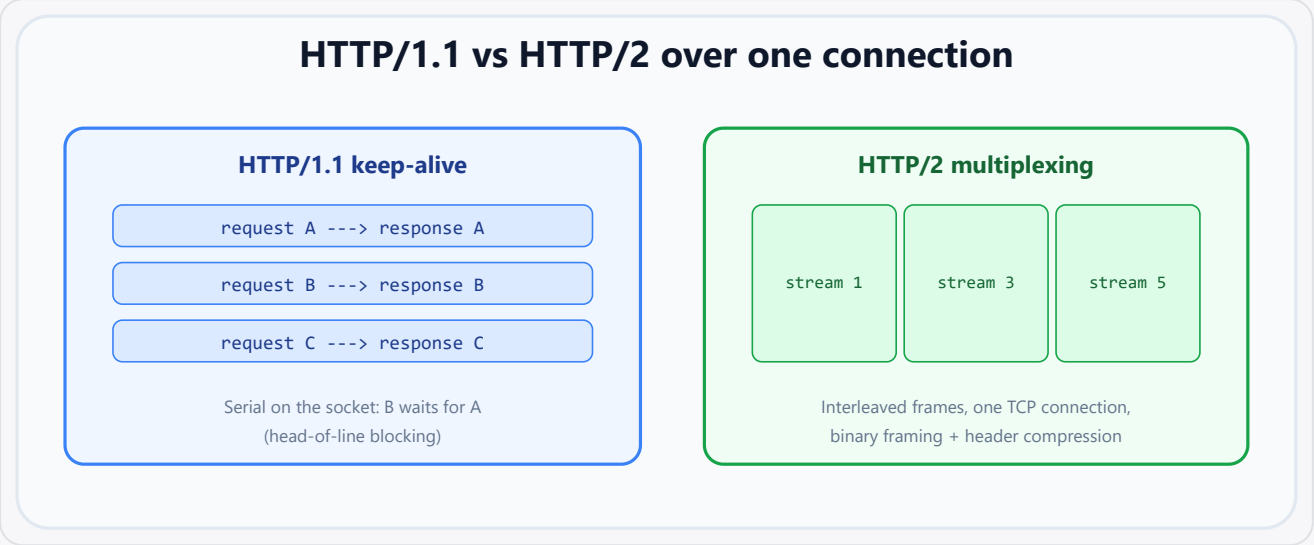
The TLS handshake. The client sends a *ClientHello* advertising the TLS versions and cipher suites it supports. The server replies with a *ServerHello* choosing a cipher and presenting its **certificate** — a document containing the server's public key, signed by a Certificate Authority (CA) the client trusts. The client verifies the signature chain up to a trusted root, then both sides run a key-agreement algorithm (modern TLS 1.3 uses ephemeral Diffie-Hellman, ECDHE) to derive a shared **session key** without ever transmitting it. From the *Finished* messages onward, all HTTP data is encrypted with that symmetric key. TLS 1.3 (the current standard) trimmed this to a single round-trip and removed legacy insecure ciphers.

Asymmetric vs symmetric. A frequent point of confusion: the certificate's public/private key pair (asymmetric, slow) is used only to *authenticate* and to safely agree on keys. The actual request/response bytes are encrypted with a *symmetric* key (fast) derived during the handshake. Asymmetric bootstraps trust; symmetric does the heavy lifting.

Serving TLS in Node. Use `node:https` (or `node:http2`) instead of `node:http`. You pass a `key` (private key) and `cert` (certificate) in the options. In real deployments you rarely terminate TLS in Node itself — a reverse proxy or load balancer (Nginx, a cloud LB, Caddy) usually terminates TLS and forwards plain HTTP to your Node process on a

Keep-alive. HTTP/1.1 defaults to persistent connections: after a response, the TCP (and TLS) connection stays open for the next request, avoiding a fresh handshake each time. This is controlled by the `Connection` header and Node's `server.keepAliveTimeout`. Without keep-alive, every request pays a full TCP + TLS setup cost.

HTTP/2. Same HTTP semantics (methods, status codes, headers) but a different wire format: binary framing, header compression (HPACK), and **multiplexing** — many concurrent requests interleaved over a single TCP connection, eliminating the head-of-line blocking of HTTP/1.1 where one slow response stalls the queue behind it. It is served in Node via `node:http2` and almost always runs over TLS (`h2` is negotiated during the handshake via ALPN).



HTTP/1.1 serialises requests on the socket; HTTP/2 interleaves independent streams, removing head-of-line blocking.

CORS. The Cross-Origin Resource Sharing rules are enforced by the *browser*, not your server. By default a page at `https://app.com` cannot read a `fetch` response from `https://api.other.com` unless the server opts in with `Access-Control-Allow-Origin`. For "non-simple" requests the browser first sends a *preflight* `OPTIONS` request; your server must answer it with the appropriate `Access-Control-Allow-*` headers or the real request is blocked. Crucially, CORS is a browser security policy — `curl` and server-to-server calls ignore it entirely.

A request is “simple” (no preflight) only if it uses `GET`, `HEAD`, or `POST`; carries no custom headers beyond a short CORS-safelisted set; and, if it has a body, uses one of three content types: `text/plain`, `application/x-www-form-urlencoded`, or `multipart/form-data`. The moment you send `Content-Type: application/json`, add an `Authorization` header, or use `PUT`/`DELETE`/`PATCH`, the request becomes “preflighted” and the browser first sends an `OPTIONS` with `Access-Control-Request-Method` and `Access-Control-Request-Headers`, expecting your server to echo permission back. You can cut the cost of repeated preflights by sending `Access-Control-Max-Age`, which tells the browser how many seconds it may cache the preflight result. And if the request needs cookies, the client must set `credentials: 'include'` *and* your server must reply with both a specific (non-wildcard) origin and `Access-Control-Allow-Credentials: true`.

SNI, ALPN, and what the handshake negotiates. Two extensions ride along in the ClientHello and are worth naming. *SNI* (Server Name Indication) is the hostname the client wants, sent in the clear so that a single IP hosting many sites can present the right certificate — it is also why the hostname is one of the few things TLS does not hide. *ALPN* (Application-Layer Protocol Negotiation) is how client and server agree, during the same handshake, on whether to speak HTTP/1.1 or HTTP/2 (`h2`); this is why HTTP/2 in practice needs no extra round-trip and why `allowHTTP1: true` lets one `http2.createSecureServer` serve both kinds of client. TLS 1.3 also enables *session resumption* and *0-RTT*, letting a returning client skip most of the handshake, which further amplifies the keep-alive and connection-reuse benefits described above.

Code examples

An HTTPS server with a key and certificate (here self-signed for local dev):

JAVASCRIPT

Copy

```
const https = require('node:https');
const fs = require('node:fs');

// Generate a dev cert:
//   openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.p
```

```
};

https.createServer(options, (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Served over TLS
');
}).listen(8443, () => console.log('https://localhost:8443'));
```

Redirecting all plain HTTP traffic to HTTPS — a standard production pattern:

JAVASCRIPT Copy

```
const http = require('node:http');

// Runs on port 80, bounces everyone to https.
http.createServer((req, res) => {
  const host = req.headers.host?.split(':')[0] ?? 'localhost';
  res.writeHead(301, { Location: `https://${host}${req.url}` });
  res.end();
}).listen(80);
```

An HTTP/2 server (over TLS, with HTTP/1.1 fallback via ALPN):

JAVASCRIPT Copy

```
const http2 = require('node:http2');
const fs = require('node:fs');

const server = http2.createSecureServer({
  key: fs.readFileSync('key.pem'),
  cert: fs.readFileSync('cert.pem'),
  allowHTTP1: true, // fall back for older clients
});

server.on('stream', (stream, headers) => {
  // http2 uses pseudo-headers: :method, :path, :authority
  stream.respond({ ':status': 200, 'content-type': 'text/plain' });
  stream.end(`hello over ${headers[':method']} ${headers[':path']}
```

```
server.listen(8443);
```

Handling CORS, including the preflight `OPTIONS` request:

JAVASCRIPT Copy

```
const http = require('node:http');

const ALLOWED = 'https://app.example.com';

http.createServer((req, res) => {
  res.setHeader('Access-Control-Allow-Origin', ALLOWED);
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorizati

  // Preflight: browser asks permission before the real request.
  if (req.method === 'OPTIONS') {
    res.writeHead(204).end();
    return;
  }

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ ok: true }));
}).listen(3000);
```

Common mistakes & gotchas

- **Thinking TLS hides everything.** The destination IP, the SNI hostname, and traffic size/timing are still observable; TLS encrypts the HTTP payload, not the fact that you connected.
- **Confusing which key encrypts the data.** The certificate's asymmetric key only authenticates and negotiates; bulk data uses the derived symmetric key.
- **Committing private keys.** A leaked `key.pem` compromises the whole site; keep keys out of source control and images.

- **Expecting the server to "fix" CORS for curl.** CORS is browser-enforced; a blocked `fetch` is not a server error, and `curl` succeeding proves nothing about it.
- **Using `Access-Control-Allow-Origin: *` with credentials.** The wildcard is rejected when `credentials: 'include'` is set; you must echo a specific origin and add `Access-Control-Allow-Credentials: true`.
- **Forgetting the preflight.** Custom headers or non-simple methods trigger an `OPTIONS` preflight; not answering it blocks the real request.
- **Disabling certificate validation** (`rejectUnauthorized: false`) to "make it work," which silently removes TLS's authentication guarantee.

Best practices

- **Serve everything over HTTPS** and 301-redirect plain HTTP; add `Strict-Transport-Security` (HSTS) to force it.
- **Use TLS 1.2+ (prefer 1.3)** and let Node's defaults pick strong cipher suites; do not hand-configure weak ones.
- **Terminate TLS at a proxy/load balancer** in production, and get certificates from an automated CA (Let's Encrypt) with auto-renewal.
- **Keep keep-alive on** and tune `keepAliveTimeout` to reuse connections and cut handshake cost.
- **Adopt HTTP/2** for many small concurrent assets to benefit from multiplexing and header compression.
- **Scope CORS narrowly**: an explicit allow-list of origins, only the methods/headers you need, and never `*` alongside credentials.
- **Set security headers** (`HSTS`, `X-Content-Type-Options`) and never disable certificate verification in clients.

⭐ Interview lens

> **Q: What does HTTPS add over HTTP, precisely?**
> It wraps HTTP in TLS, providing confidentiality (encryption), integrity (tamper detection), and authentication (a CA-signed certificate proves the server's

identity). The HTTP semantics are unchanged; only the transport is now encrypted and authenticated.

Q: Walk through the TLS handshake at a high level.

ClientHello (offered versions/ciphers) → ServerHello + certificate (server's public key) → client verifies the cert chain and both derive a shared session key via ephemeral Diffie-Hellman → Finished messages → encrypted application data flows over the symmetric session key. TLS 1.3 does this in one round-trip.

Q: Trap — is the request body encrypted with the server's public key?

No. The public/private (asymmetric) key pair is used only to authenticate the server and to agree on a shared secret. The actual data is encrypted with a fast *symmetric* session key derived during the handshake. Symmetric ciphers are far faster for bulk data.

Q: What problem does HTTP/2 multiplexing solve?

HTTP/1.1 head-of-line blocking: on a single connection, request B must wait for A's response. HTTP/2 interleaves multiple independent streams over one TCP connection using binary framing, so responses can arrive out of order and no single slow response stalls the rest. It also compresses headers (HPACK).

Q: A browser `fetch` is blocked by CORS but `curl` works. Whose fault, and how do you fix it?

Neither is "broken" — CORS is a browser-enforced policy that `curl` ignores. The fix is on the server: respond with `Access-Control-Allow-Origin` (a specific origin, or `*` for public non-credentialed APIs) and, for non-simple requests, answer the `OPTIONS` preflight with the matching `Allow-Methods` / `Allow-Headers`.

It reuses one TCP + TLS connection for many requests, so you pay the connection setup and TLS handshake cost once instead of per request. Over HTTPS the handshake is the expensive part, so keep-alive (and connection pooling on the client side) is a major latency win.

Cheat-card

- HTTPS = HTTP inside TLS → confidentiality + integrity + authentication.
- Handshake: ClientHello → cert → key agreement → encrypted data. TLS 1.3 = 1-RTT.
- Asymmetric keys authenticate + agree; symmetric session key encrypts the data.
- Node: `node:https` / `node:http2` with `{ key, cert }`.
- Keep-alive reuses the connection → skips repeat handshakes.
- HTTP/2: binary framing + HPACK + multiplexing (no head-of-line blocking).
- CORS is browser-enforced; server sends `Access-Control-Allow-*`; answer `OPTIONS` preflight.
- No `*` origin with credentials; never disable cert validation.

QUICK CHECK — no marks, just for you.

1. What three guarantees does TLS add to HTTP?

- ○ Confidentiality (encryption), integrity (tamper detection), and authentication (a CA-signed certificate proves the server identity)
- ○ Compression, caching, and connection multiplexing
- ○ Availability, load balancing, and rate limiting
- ○ Faster responses, lower latency, and reduced bandwidth

Check answer

- Yes; every request byte is encrypted with the certificate's public key
- No; the asymmetric key pair only authenticates the server and agrees on a shared secret, and the actual data uses a fast symmetric session key
- Yes, but only until the first response, then it switches to symmetric
- No; the body is never encrypted, only the headers are

Check answer

3. A browser fetch is blocked by CORS but the same request works with curl. Where is the fix?

- In the browser: disable CORS with a client-side flag
- In curl: it is silently corrupting the request, so switch to fetch
- On the server: CORS is browser-enforced (curl ignores it), so respond with Access-Control-Allow-Origin and answer any OPTIONS preflight
- Nowhere: a CORS block means the server is genuinely down

Check answer

Node.js Backend Kit

LEARN MORE

ARTICLES & DOCS

- HTTPS | Node.js Documentation
  nodejs.org
- TLS (SSL) | Node.js Documentation
  nodejs.org
- Transport Layer Security (TLS) | MDN
  developer.mozilla.org
- Evolution of HTTP | MDN
  developer.mozilla.org

VIDEOS



The TLS Handshake Explained with Example (the Math)
Hussein Nasser

Hand-picked references for this lesson.